

연락처 관리 웹 서비스 — 기능 정의서 (2차 과제)

항목	내용
문서명	연락처 관리 웹 서비스(2차 과제) 기능 정의서
문서 유형	Function Definition (Functional Specification)
과제 구분	2차 과제 — FastAPI + DB 연락처 프로그램
선행 과제	1차 과제 — 콘솔 연락처 관리 프로그램
버전	v1.0
언어/환경	Python 3.12+ / FastAPI 0.139.0 / SQLAlchemy 2.0.51 / PostgreSQL 16
상태	확정(Baseline)

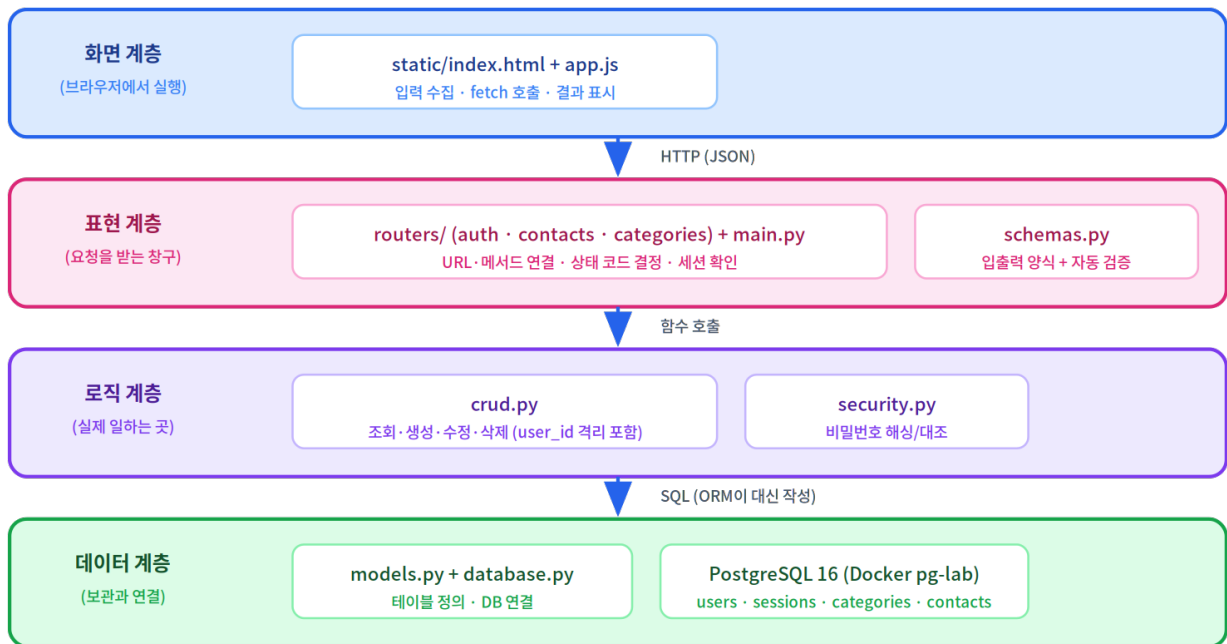
이 문서의 역할(안내) 프로그램을 구성하는 각 기능을 파일(모듈)과 함수 단위로 정의합니다. 구현요구사항(01 문서)이 "무엇을"이라면, 본 문서는 "그 일을 어느 파일의 어떤 함수가 하는가"를 다룹니다. 1차 과제는 파일 1개 (`member_manager.py`)에 함수 13개였지만, 2차 과제는 **역할별로 파일을 나눕니다** — 이 "나누는 기준"을 이해하는 것이 본 문서의 절반입니다.

1. 프로젝트 구조 (파일 아키텍처)

1-1. 폴더/파일 구성

contact_app/	
├── main.py	# 앱 조립: FastAPI 생성, 라우터 등록, 화면 제공
├── database.py	# DB 연결: 엔진, DB세션 공장, get_db
├── models.py	# 테이블 정의: User, LoginSession, Category, Contact
├── schemas.py	# 입출력 양식: Pydantic 모델 (검증 규칙의 집)
├── security.py	# 비밀번호 해싱: hash_password, verify_password
├── crud.py	# DB 작업 함수: 조회/생성/수정/삭제 (핵심 로직)
├── routers/	
│ ├── auth.py	# 인증 엔드포인트 4개 + get_current_user 의존성
│ ├── contacts.py	# 연락처 엔드포인트 4개
│ └── categories.py	# 카테고리 엔드포인트 4개
└── static/	
├── index.html	# 화면 (02 문서의 SCR-001~003)
└── app.js	# 화면 동작 (fetch 호출, DOM 갱신)

1-2. 왜 이렇게 나누나 — 계층 아키텍처



비유로 이해하기 — "식당의 부서": 1차 과제는 사장 혼자 주문 · 조리 · 창고를 다 하는 가게였습니다(파일 1개). 2차 과제는 **홀(routers) — 주방(crud) — 창고(models/DB)**로 부서를 나눈 식당입니다. 홀 직원은 요리하지 않고(라우터에 DB 코드 금지), 주방장은 손님을 직접 상대하지 않습니다(crud에 상태 코드 결정 금지). 부서를 나누는 이유는 1차 때와 같습니다 — **고칠 때 한 곳만 고치기 위해서**. 나중에 화면을 앱으로 바꿔도 주방(crud)은 그대로 씁니다.

1-3. 용어 주의 — "세션"이 두 가지다 (초보자 최다 혼동 지점)

2차 과제에는 이름이 같은 별개 개념이 두 개 등장합니다. 반드시 구분하세요.

구분	정체	어디서 오나	코드에서의 이름
DB 세션	파이썬 ↔ DB 사이의 대화 연결 (요청마다 열고 닫음)	SQLAlchemy	<code>Session</code> , <code>db: Session = Depends(get_db)</code>
로그인 세션	"누가 로그인 중인가"를 적어 둔 장부의 행	우리가 만든 <code>sessions</code> 테이블	모델 클래스 <code>LoginSession</code>

이 충돌 때문에 로그인 세션의 모델 클래스 이름을 `Session` 이 아니라 `LoginSession` 으로 정합니다(테이블명은 `sessions` 그대로). `from sqlalchemy.orm import Session` 과 이름이 겹치면 임포트 순서에 따라 한쪽이 덮여 찾기 어려운 버그가 나기 때문입니다. "이름 충돌을 이름 짓기로 피한다" — 실무에서 매우 자주 쓰는 기법입니다.

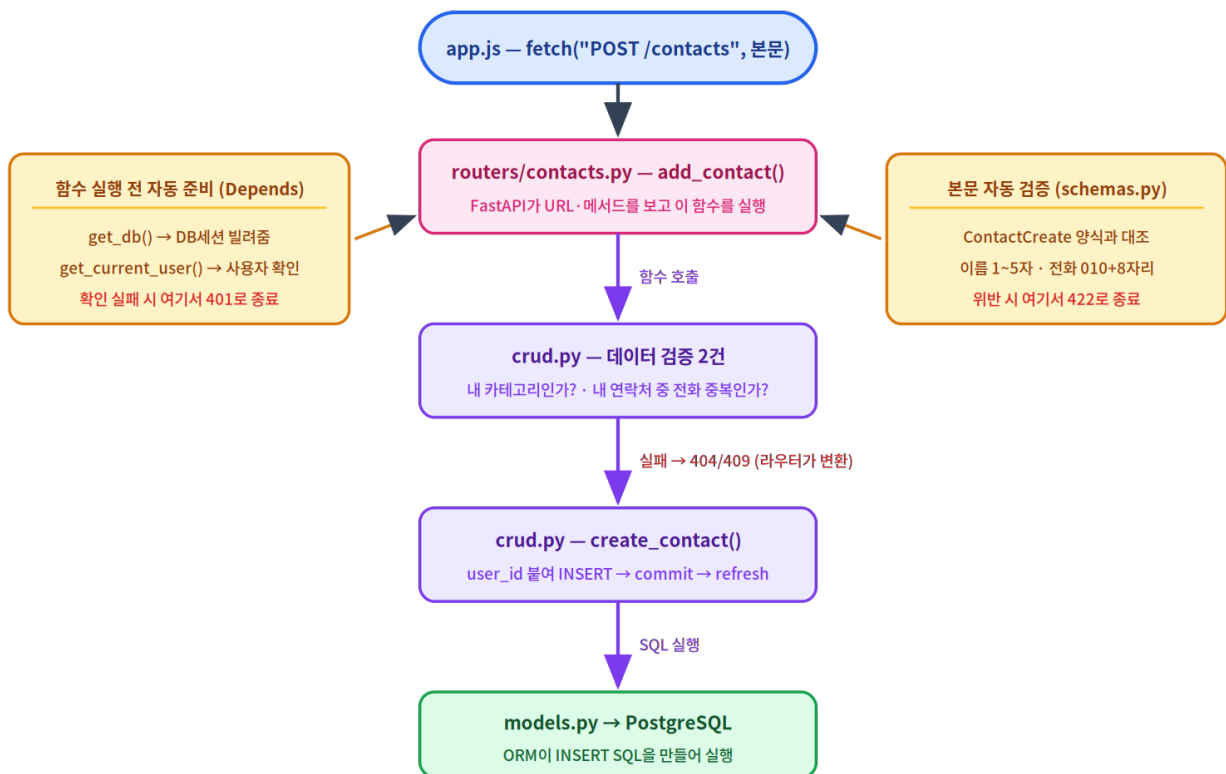
2. 기능 목록 (Function Inventory)

기능 ID	기능명	위치(파일)	대표 함수/객체	관련 FR
FN-001	DB 연결과 DB세션 공급	database.py	<code>engine</code> , <code>get_db()</code>	전체
FN-002	테이블 모델 정의	models.py	<code>User</code> , <code>LoginSession</code> , <code>Category</code> , <code>Contact</code>	§ 1 데이터 모델
FN-003	입출력 양식 + 자동 검증	schemas.py	<code>SignupIn</code> , <code>ContactCreate</code> , <code>ContactUpdate</code> , <code>ContactOut</code> 등	유효성 규칙
FN-004	비밀번호 해싱/대조	security.py	<code>hash_password()</code> , <code>verify_password()</code>	FR-01, FR-02
FN-005	회원가입 처리	crud.py + routers/auth.py	<code>create_user()</code> (+기본 카테고리 시드)	FR-01
FN-006	로그인·세션 발급	crud.py + routers/auth.py	<code>authenticate_user()</code> , <code>create_login_session()</code>	FR-02
FN-007	현재 사용자 확인 (의존성)	routers/auth.py	<code>get_current_user()</code>	FR-04 + 모든 보호 API
FN-008	로그아웃·세션 삭제	crud.py + routers/auth.py	<code>delete_login_session()</code>	FR-03
FN-009	연락처 CRUD	crud.py	<code>list_contacts()</code> , <code>create_contact()</code> , <code>get_my_contact()</code> , <code>update_contact()</code> , <code>delete_contact()</code>	FR-05~08
FN-010	카테고리 CRUD + 사용 중 확인	crud.py	<code>list_categories()</code> , <code>create_category()</code> , <code>update_category()</code> , <code>delete_category()</code> , <code>count_contacts_in_category()</code>	FR-09~12
FN-011	엔드포인트 정의 (창구)	routers/ 3개 파일	<code>@router.get/post/patch/delete</code> 함수 12개	FR-01~12
FN-012	화면 제공 + 화면 동작	main.py + static/	<code>GET /</code> → index.html, app.js	FR-13
FN-013	예외 → 상태 코드 변환	routers/ 전반	<code>HTTPException(status_code, detail)</code>	NFR-01

1차 과제와의 대응: 1차의 `load_data/save_data` 는 FN-001(DB 연결)로, `validate_*` 3형제는 FN-003(schemas)으로, `find_by_name` 은 FN-009의 `list_contacts(name=...)` 로 흡수됩니다. 완전히 새로운 것은 **FN-004(해싱)**, **FN-006~008(세션)**, **FN-007(현재 사용자 확인)** 뿐입니다.

3. 전체 워크플로우 — 요청 1건의 함수 호출 관계

"연락처 추가" 요청 1건이 어떤 파일의 어떤 함수를 순서대로 거치는지의 전체 지도입니다. (1차 과제 문서의 "기능 아키텍처 — 함수 호출 관계"에 대응)



응답은 역순으로: PostgreSQL → crud(연락처 객체) → 라우터(ContactOut 양식으로 변환) → 201 JSON → app.js(목록 새로 고침)

초보자 포인트 — Depends(의존성 주입): 라우터 함수의 매개변수에 `db: Session = Depends(get_db)` , `user: User = Depends(get_current_user)` 라고 적으면, FastAPI가 함수를 실행하기 전에 준비물을 자동으로 챙겨서 넣어 줍니다. 요리사(함수)가 일 시작 전에 조수(FastAPI)가 재료(DB세션)와 주문서 확인(로그인 사용자)을 끝내 주는 것과 같습니다. 1차 과제에서 `main` 루프가 매번 직접 하던 준비 작업이, 2차에서는 선언 한 줄로 자동화됩니다. 준비에 실패하면(로그인 안 됨) 본 함수는 **아예 실행되지 않고** 401이 나옵니다 — 그래서 12개 API 어디에도 "로그인 확인 코드"를 반복해서 쓸 필요가 없습니다.

4. 기능 상세 정의

FN-001 · DB 연결과 DB세션 공급 (database.py)

항목	내용
구성	<code>engine</code> (연결 담당), <code>SessionLocal</code> (DB세션 공장), <code>Base</code> (모델 부모), <code>get_db()</code> (의존성)
<code>get_db()</code> 동작	요청마다 DB세션을 열어 빌려주고(<code>yield</code>), 요청이 끝나면 반드시 닫음(<code>finally</code>)
접속 문자열	<code>postgresql+psycopg://</code> 로 시작 (드라이버 psycopg 3) — 상세는 05 TRD
1차 대응	<code>load_data</code> / <code>save_data</code> 가 하던 "저장소 연결" 역할
예외	DB(pg-lab)가 꺼져 있으면 연결 오류 → 실행 전 Docker 기동 확인

FN-002 · 테이블 모델 정의 (models.py)

항목	내용
클래스	<code>User</code> , <code>LoginSession</code> (§ 1-3 참조), <code>Category</code> , <code>Contact</code> — 01 문서 § 1-2의 4개 테이블과 1:1
문법	SQLAlchemy 2.0 방식 <code>Mapped[...] + mapped_column(...)</code>
제약 표현	<code>unique=True</code> (username), <code>ForeignKey(...)</code> , 복합 UNIQUE는 <code>UniqueConstraint("user_id", "phone")</code>
출력	클래스 정의가 곧 테이블 설계도 — 앱 시작 시 테이블 자동 생성 (방법은 05 TRD)

FN-003 · 입출력 양식 + 자동 검증 (schemas.py)

스키마	용도	담는 규칙 (01 문서 § 4-1과 동일)
<code>SignupIn</code>	가입/로그인 입력	username 4~20자 · <code>^[a-z0-9]+\$</code> / password 4~20자
<code>UserOut</code>	사용자 응답	id, username만 — <code>password_hash</code> 는 절대 포함 금지
<code>ContactCreate</code>	연락처 추가 입력	name 1~5자 / phone <code>^010\d{8}\$</code> / addr 자유 / category_id
<code>ContactUpdate</code>	연락처 수정 입력	위와 같은 규칙, 단 모든 필드가 선택(<code> None</code>) — 보낸 것만 수정(PATCH)
<code>ContactOut</code>	연락처 응답	id, name, phone, addr, category_id, category_name
<code>ContactListOut</code>	목록 응답	total, items(ContactOut 배열) — 1차의 "총 N명" 계승
<code>CategoryCreate/Update</code>	카테고리 입력	name 1~10자
<code>CategoryOut</code>	카테고리 응답	id, name

입력용과 출력용을 왜 나누나: 입력 양식에 id가 있으면 사용자가 id를 마음대로 지정해 보낼 수 있고, 출력 양식에 `password_hash`가 있으면 해시가 응답에 노출됩니다. "받을 것"과 "보여줄 것"은 항상 다른 양식으로 — 실무 API 설계의 철칙입니다.

FN-004 · 비밀번호 해싱/대조 (security.py)

함수	입력	출력	설명
<code>hash_password(원문)</code>	비밀번호 원문	해시 문자열	pwdlib(Argon2)로 변환 — 가입 시 1회
<code>verify_password(원문, 해시)</code>	로그인 입력값, 저장된 해시	True/False	로그인 때마다 대조

파일을 따로 두는 이유: "비밀번호 다루는 코드가 여기 다 있다"를 보장하기 위해서입니다. 나중에 해싱 방식을 바꿔도 이 파일만 고치면 됩니다.

FN-005 · 회원가입 처리 (crud.py `create_user`)

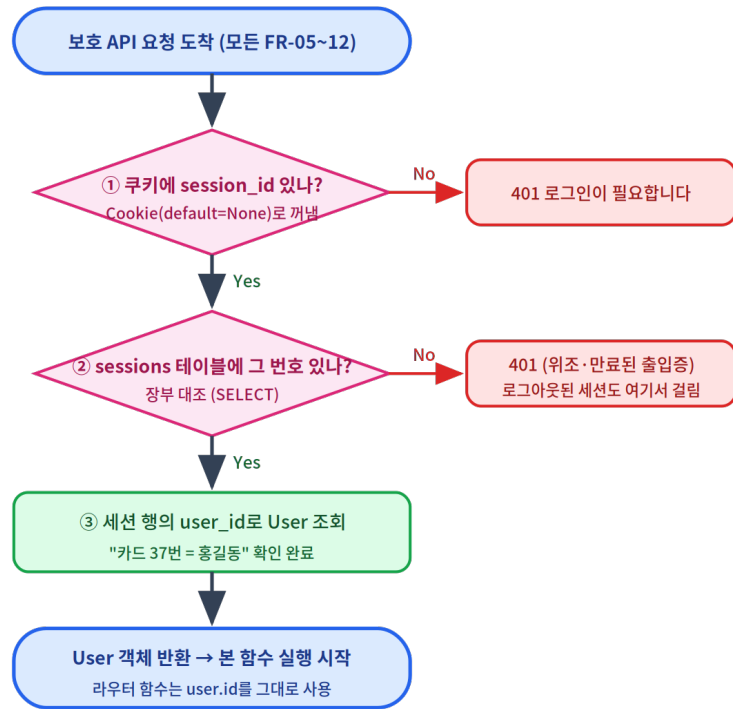
항목	내용
처리 순서	① username 중복 확인(있으면 라우터가 409) ② <code>hash_password</code> 로 해시 ③ User INSERT ④ 같은 커밋 안에서 기본 카테고리 3개(가족/친구/기타) INSERT ⑤ commit
왜 같은 커밋인가	사용자만 만들어지고 카테고리 생성이 실패하면 "카테고리 없는 반쪽 계정"이 남음 → 한 트랜잭션으로 묶으면 둘 다 되거나 둘 다 안 되거나 만 존재 (1차 과제에는 없던, DB만이 주는 안전장치)

FN-006 · 로그인·세션 발급 (crud.py + routers/auth.py)

함수	처리
<code>authenticate_user(db, username, password)</code>	사용자 조회 → <code>verify_password</code> 대조 → 성공 시 User, 실패 시 None (없는 아이디/틀린 비번 구분 없이 — 01 문서 보안 규칙)
<code>create_login_session(db, user_id)</code>	<code>secrets.token_hex(32)</code> 로 세션 번호 생성 → sessions INSERT + commit → 번호 반환
라우터 마무리	<code>response.set_cookie("session_id", 번호, httponly=True)</code> — httponly는 JavaScript가 쿠키를 읽지 못하게 하는 보호 옵션

FN-007 · 현재 사용자 확인 — `get_current_user()` ★ 2차 과제의 심장

이 함수 하나가 보호 API 8개 전부의 문지기입니다. 단계별 흐름:



항목	내용
시그니처	<code>get_current_user(session_id: str None = Cookie(default=None), db: Session = Depends(get_db)) -> User</code>
사용법	보호할 라우터 함수에 <code>user: models.User = Depends(get_current_user)</code> 한 줄 추가 — 끝
1차 대응	(없음 — 2차의 완전 신규 개념이자, 멀티 유저를 가능하게 하는 단 하나의 함수)

FN-008 · 로그아웃 (crud.py delete_login_session)

항목	내용
처리	sessions에서 해당 행 DELETE + commit → 라우터가 <code>response.delete_cookie("session_id")</code>
결과	같은 쿠키로 다시 요청해도 FN-007 ②에서 걸려 401 — "장부에서 지우면 카드는 즉시 무효"

FN-009 · 연락처 CRUD (crud.py)

모든 함수의 첫 번째 규칙: 조회 조건에 `user_id` 가 반드시 들어간다.

함수	입력	처리	출력
<code>list_contacts(db, user_id, name=None, category_id=None)</code>	검색어(선택)	내 연락처 SELECT (+이름/카테고리 필터)	Contact 리스트
<code>get_my_contact(db, user_id, contact_id)</code>	대상 id	<code>WHERE id=? AND user_id=?</code> 로 조회	Contact 또는 None(→라우터가 404)
<code>create_contact(db, user_id, data)</code>	ContactCreate	user_id 붙여 INSERT + commit + refresh	생성된 Contact
<code>update_contact(db, contact, data)</code>	대상 + ContactUpdate	<code>model_dump(exclude_unset=True)</code> 로 보낸 항목만 갱신 + commit	수정된 Contact
<code>delete_contact(db, contact)</code>	대상	DELETE + commit	없음

`get_my_contact` 가 데이터 격리의 전부입니다. 조건에 `user_id` 가 함께 들어가므로, 남의 연락처는 id가 맞아도 조회 결과가 None → 라우터는 "없다(404)"고만 답합니다. 별도의 권한 검사 코드가 필요 없는, 가장 단순하고 안전한 구조입니다.

FN-010 · 카테고리 CRUD (crud.py)

연락처와 같은 패턴 + 함수 하나 추가:

함수	역할
<code>count_contacts_in_category(db, user_id, category_id)</code>	그 카테고리 소속 연락처 수를 셈 — 삭제 전 필수 호출, 1건 이상이면 라우터가 409(건수를 detail에 포함)

FN-011 · 엔드포인트 정의 (routers/ — 창구 12개)

라우터 함수는 "접수 → 확인 → 주방 호출 → 응답 포장" 만 합니다. 표준형:

```
# 모든 라우터 함수의 공통 골격 (연락처 추가 예)
@router.post("", response_model=schemas.ContactOut, status_code=201)
def add_contact(
    data: schemas.ContactCreate,          # 본문 → 자동 검증(422)
    user: models.User = Depends(get_current_user),  # 문지기(401)
    db: Session = Depends(get_db),        # DB세션 준비
):
    ... # 데이터 검증(404/409) 후 crud 호출, 결과 반환
```

파일	담는 창구	개수
routers/auth.py	signup, login, logout, me	4
routers/contacts.py	목록/검색, 추가, 수정, 삭제	4
routers/categories.py	목록, 추가, 수정, 삭제	4

FN-012 · 화면 제공 + 화면 동작 (main.py + static/)

항목	내용
main.py	FastAPI 앱 생성, 라우터 3개 등록, GET / → index.html 반환
index.html	02 문서의 SCR-001~003 구조 그대로
app.js	02 문서 § 5의 단계별 워크플로우(me → categories → contacts) 구현 + 버튼 이벤트

FN-013 · 예외 → 상태 코드 변환 (라우터 전반)

상황	코드 패턴	결과
crud가 None 반환 (없거나 남의 것)	<code>raise HTTPException(404, "해당 연락처가 없습니다")</code>	404 + detail
중복 발견	<code>raise HTTPException(409, "이미 등록된 전화번호입니다")</code>	409 + detail
사용 중 카테고리 삭제	<code>raise HTTPException(409, f"...연락처가 {n}건 있어 삭제할 수 없습니다...")</code>	409 + detail
형식 위반	(코드 없음 — Pydantic 자동)	422
미로그인	(코드 없음 — FN-007이 처리)	401

5. 기능 우선순위 및 구현 순서 (권장)

1차 과제의 "골격 먼저, 견고성 마지막" 원칙 그대로입니다. 00 문서의 학습 6단계와 일치합니다.

순 서	구현 대상 (FN)	완료 확인	이유
1	FN-001 DB 연결 + FN-002 모델	테이블 4개 생성 확인 (<code>psql</code>)	보관소가 있어야 나머지를 붙임
2	FN-003 스키마 + FN-004 해싱	(다음 단계에서 함께 확인)	창구 · 주방의 공용 부품
3	FN-005~008 인증 4종 + FN-007 문 지기	<code>/docs</code> 에서 가입→로그인→me→로 그아웃	문지기가 있어야 이후 모든 API를 보호 상 태로 개발
4	FN-009 + contacts 라우터	<code>/docs</code> 에서 CRUD + 계정 2개 격리 확인	핵심 데이터 먼저
5	FN-010 + categories 라우터	사용 중 삭제 409 확인	같은 패턴 반복 + 엡지 케이스
6	FN-012 화면	브라우저에서 전 기능 동작	API가 다 검증된 뒤 화면을 엡음
7	FN-013 예외 마감	01 문서 § 5-3 목록 전부 통과	견고성 마무리

6. 기능 점검 체크리스트

- [] FN-001: pg-lab이 켜진 상태에서 서버가 오류 없이 시작
- [] FN-002: users·sessions·categories·contacts 테이블 4개 생성 확인
- [] FN-005: 가입 직후 그 사용자의 카테고리가 정확히 3개(가족/친구/기타)
- [] FN-006: 로그인 응답에 Set-Cookie가 있고, 이후 요청에 쿠키가 자동 첨부됨
- [] FN-007: 쿠키 없이/영터리 쿠키로 보호 API 호출 시 401
- [] FN-008: 로그아웃 후 같은 쿠키로 요청 시 401
- [] FN-009: 계정 2개로 서로의 연락처가 보이지 않음 + 남의 id 접근 시 404
- [] FN-009: 같은 번호 두 번 추가 시 409, PATCH는 보낸 항목만 바뀜
- [] FN-010: 소속 연락처가 있는 카테고리 삭제 시 409 + 건수 안내
- [] FN-011: /docs 에 창구 12개가 전부 보이고 실행 가능
- [] FN-012: 브라우저 화면에서 로그인부터 관리까지 전 기능 동작
- [] FN-013: 어떤 요청에도 500이 나지 않음

7. 기능-요구사항 매핑

FN	FR	SCR (02 문서)	00 문서 학습 목표
FN-001, 002	§ 1 데이터 모델	-	3-2 테이블, 3-7 FK
FN-003	§ 4 유효성	-	3-4 자동 검증
FN-004~008	FR-01~04	SCR-001	3-6 세션 로그인
FN-009	FR-05~08	SCR-002	3-1 REST API
FN-010	FR-09~12	SCR-003	3-7 카테고리(FK)
FN-011	FR-01~12	전체	3-1 REST API
FN-012	FR-13	SCR-001~003	3-5 프론트-백 연동
FN-013	NFR-01	SCR-900	평가 관점(예외)